laneproof: admission control and a merged-result evidence gate for parallel coding agents

Abhay Mettu
https://github.com/abhaymettu/laneproof

November 2026

Abstract

Running several coding agents at once is now routine, and the usual tools give each agent its own git worktree and leave the rest to a human. Two things go wrong. Agents report "done" for work that is not done, and work that passes on its own branch fails once it is combined with what other agents landed. laneproof is a small coordinator built around three mechanisms: an admission controller whose reservations each admit exactly one agent start, receipts in which every claim is labelled FACT, CLAIM or HYPOTHESIS and only FACTs (commands with expected results) count, and a gate that merges each lane onto the latest main branch in a scratch worktree and re-runs that evidence there before main moves. Because nothing lands unchecked, the gate can resolve textual conflicts optimistically with a union merge. We compare four ways of running the same tasks with Claude Code: one agent at a time, four agents in one checkout, four agents in worktrees merged on their say-so, and laneproof. Over 84 lanes per condition (Claude Sonnet 5.5 and Haiku 5.5, three runs per cell), laneproof's status matched the hidden ground truth for every lane, while agents' own reports were wrong for 16 lanes in a shared checkout and 20 in worktrees. A shared checkout finished the most tasks (84 against laneproof's 73) in about a third of the time.

1 Problem

A coordinator that hands whole tasks to parallel agents has to answer three questions: may another agent start now, how do agents avoid trampling each other, and when is a task actually done. Current tools answer the second question with isolation (git worktrees in claude-squad and Conductor, containers in Sculptor) and leave the first and third to the operator. Recent work measures what the third costs. CooperBench [1] reports that agents make claims about code state they cannot verify, and "Passes Alone, Fails Together" [2] shows that patches which merge cleanly can still fail hidden tests together. Geng and Neubig [3] coordinate asynchronous agents through git worktrees, commits and merges. Single-agent completion gates exist (Claude Code's `TaskCompleted` hook for agent teams, and small open-source gates that require a receipt before an agent may stop), but they check one agent's claim against its own tree.

laneproof distils a harness the author runs their own work through into a general tool, and the question it tests is narrow: what does it cost, and what does it buy, to make "re-run the evidence on the merged result" the only way work reaches main?

2 Design

Admission. `reserve` takes one file lock, counts held reservations plus any live lane process that no held reservation covers (occupancy is observed, not trusted), and checks a cap, free disk, load per CPU and an optional quota command. Anything it cannot read refuses. `claim` turns a reservation into the right to start one agent by creating a start marker with `O_EXCL` before any process exists, so a coordinator that crashes and replays its start cannot launch a second agent. In a test, twenty concurrent `reserve` calls at cap 4 produce exactly four grants.

Lanes and receipts. Each lane is a worktree on its own branch. A lane finishes by writing a receipt whose claims carry labels: FACT (a shell command, an expected exit status and optionally a string the output must contain), CLAIM (unchecked) or HYPOTHESIS. Every acceptance criterion in the task must be covered by at least one FACT. A lane that cannot continue writes status `asking` with a question instead of guessing.

The gate. Holding a landing lock, the gate checks coverage and scope (globs the lane may and may not change), merges the lane onto the current main in a scratch worktree, re-runs every FACT and the coordinator's own check there, and only then advances main by compare-and-swap. A failure goes back to the lane with the command and its output; the lane gets at most two retries. Landing is keyed by the receipt and the lane's commit, so replaying a verdict changes nothing.

Optimistic merging. When the merge conflicts, the gate first resolves every conflicting hunk by keeping both sides (`git merge-file --union`) in its scratch tree. That would be reckless without the gate; with it, a bad union merge simply fails a re-run check and the lane gets the conflict to resolve by hand.

3 Benchmark

Tasks. The fixture is `textkit`, a small Python CLI package with a command registry, a README command table checked against the registry by a test, and a changelog. Eight feature tasks each add a module, a CLI subcommand, a README row, a changelog bullet and tests (six acceptance criteria each). Every task touches the same three shared files, as real feature work does. The *coupled* suite adds two cross-cutting tasks: *docsync* (every README cell must equal its command's help string, enforced by a new test) and *jsonout* (a global `--json` flag that must work for every command). Ground truth is a hidden checker that agents never see; every assertion in it maps to a stated criterion. Reference solutions pass all ten hidden checks and the bare fixture fails all of them.

Conditions. All conditions run `claude -p` with the same task text, isolated from the operator's settings, hooks, MCP servers and memory, with Bash, Read, Edit and Write tools. *seq*: one agent at a time in one checkout. *shared*: four at once in one checkout. *worktree*: four at once, one worktree each, each branch merged into main when its agent says done; on a merge conflict the agent is resumed to merge main and resolve it, up to two times, as an operator of a worktree tool would do by hand. *laneproof*: cap 4, receipts, the gate with union merging, up to two retries. A strict variant without union merging is reported as an ablation.

Measures. *Truly done*: the hidden check passes on the final main. *Said done, isn't*: the agent reported done (for laneproof: the gate landed it) and the hidden check fails on final main. *Said not done, is*: the reverse. *Main green*: the repository's own test suite passes at the end. *Gate bounces* and, of those, how many lanes were wrong on their own branch according to the hidden checks. Wall clock, turns and tokens (including cache reads) come from the stream-json logs.

4 Results

model	condition	runs	truly done	said done, isn't	main green	bounces (wrong)	wall (s)	tokens
haiku, coupled suite								
	sequential	3	29/30	1	3/3	–	326	
	parallel, shared tree	3	30/30	0	3/3	–	144	
	parallel, worktrees	3	25/30	5	3/3	–	171	
	laneproof	3	26/30	0	3/3	26 (0)	404	
sonnet, coupled suite								
	sequential	3	30/30	0	3/3	–	216	
	parallel, shared tree	3	30/30	0	3/3	–	174	
	parallel, worktrees	3	23/30	7	3/3	–	115	
	laneproof	3	24/30	0	3/3	36 (1)	437	
sonnet, independent suite								
	sequential	3	24/24	0	3/3	–	148	
	parallel, shared tree	3	24/24	0	3/3	–	69	
	parallel, worktrees	3	16/24	8	3/3	–	102	
	laneproof, strict merge	1	7/8	0	1/1	12 (0)	810	
	laneproof	3	23/24	0	3/3	24 (0)	201	

Table 1: All runs, scored by `python3 bench/score.py` from the raw logs in `bench/runs/`. Bounces are done-claims the gate sent back; the number in parentheses is how many of those lanes also failed the hidden checks on their own branch.

Reports. laneproof made no wrong report in 84 lanes: every landed task passed the hidden checks and every unlanded task failed them. Self-reports were wrong for 1 lane run sequentially, 16 lanes in a shared checkout (agents said "blocked" when a neighbour's unfinished work had broken the suite, and the task was done at the end) and 20 lanes in worktrees (agents said "done", and 18 of those branches passed the hidden checks alone but not on the final main, mostly because repeated conflicts kept them from merging).

Throughput and cost. The shared checkout completed all 84 tasks: Claude Code refuses to edit a file that changed since it was read, which prevented clobbering here. laneproof completed 73; the other 11 used up their two retries, mostly on conflicts. Mean wall clock per run was 129 s shared, 129 s worktrees, 230 s sequential and 347 s laneproof; tokens were about twice those of the shared checkout.

What the gate catches. Of 86 bounces, one lane was wrong on its own branch by the hidden checks; the rest were conflicts or missing evidence. The gate's value in this benchmark is integration,

not catching bad code. The strict gate without union merging took 810 s for eight tasks in its one run; with union merging checked by the gate, the same suite averaged 201 s.

5 Limitations

The gate re-runs the lane's own evidence and the coordinator's check; code that is wrong but passes both lands, and the hidden checks exist to measure exactly that gap. Serialized landing and re-running tests cost wall-clock time. Under heavy contention on the same lines, a lane can exhaust its retries while main keeps moving. The benchmark is small: one fixture repository, eight or ten tasks, four lanes, two models and three runs per cell, on a machine shared with other work, so wall-clock numbers are noisy. Reservations are exactly-once only within one repository on one machine. Union merging is safe only because nothing lands unchecked; with a weak coordinator check it can land a bad merge. Running the benchmark also surfaced four bugs in the harness and the tool, all fixed before the reported runs: a shared prompt file in the generic adapter, `claude -p` processes that stay alive for minutes after their final result event while a background job they started hangs, `killpg` raising `EPERM` on macOS for a group whose leader has exited, and replayed commands inheriting the caller's standard input. Runs affected by the second bug are kept in `bench/runs-excluded/` and not reported.

6 Availability

Code, tasks, hidden checks, reference solutions, raw redacted logs and the scorer are at https://github.com/abhaymettu/laneproof. `laneproof demo` runs without a model; the benchmark reruns with `bench/all.sh MODEL REPS PARALLEL SUITE` followed by `python3 bench/score.py`.

References

[1] A. Khatua, H. Zhu, P. Tran, A. Prabhudesai et al. CooperBench: Why Coding Agents Cannot be Your Teammates Yet. arXiv:2601.13295, 2026.

[2] H. Xia, E. Wu and Y. Park. Passes Alone, Fails Together: Benchmarking Semantic Coordination in Parallel LLM-Agent Development. arXiv:2609.25396, 2026.

[3] J. Geng and G. Neubig. Effective Strategies for Asynchronous Software Engineering Agents. arXiv:2603.21489, 2026.